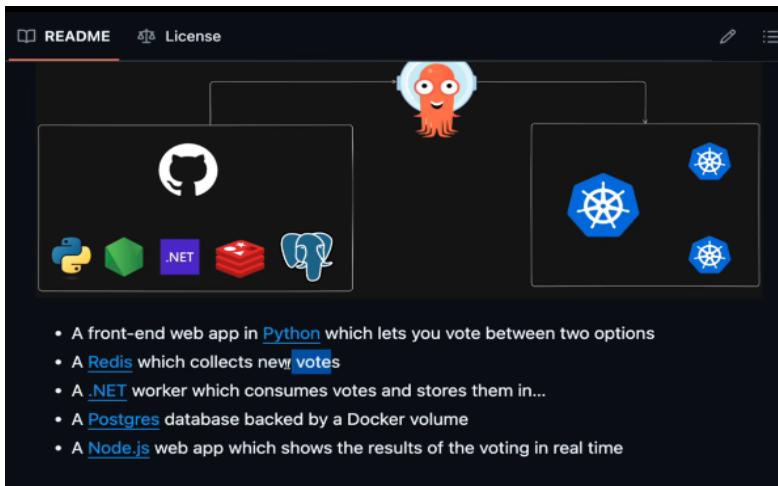
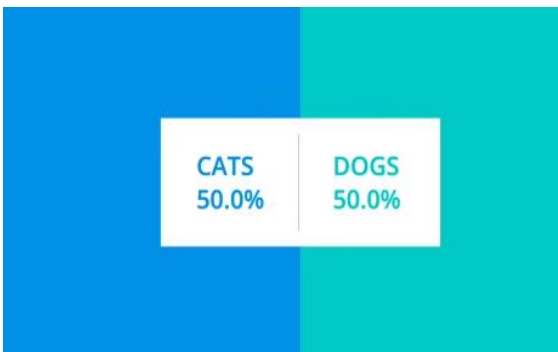


## K8's + Argo CD [trainwiths] | Kind



### CAT / DOGS Voting



Name: Kubernetes-cluster

Ubuntu

T2medium (we create multi node cluster it require more RAM)

Key pair: devops-project-k8's

Storage : 15 GB

Instance : 1

Connect instance :

```
shubham@Shubhams-MBP Downloads % chmod 400 "devops-live-project-k8s.pem"
shubham@Shubhams-MBP Downloads % ssh -i "devops-live-project-k8s.pem" ubuntu@ec2-3-21-232-239.us-east-2.compute.amazonaws.com
```

System Update :

```
ubuntu@ip-172-31-25-2: ~$ sudo apt-get update
Hit:1 http://us-east-2.ec2.archive.ubuntu.com/ubuntu no
Get:2 http://us-east-2.ec2.archive.ubuntu.com/ubuntu no
```

## Setup kubernetes cluster:

For that we need Docker because we use KIND

### Docker Install:

```
ubuntu@ip-172-31-25-2:~$ sudo apt-get install docker.io
Reading package lists... Done
Building dependency tree... 50%
```

```
ubuntu@ip-172-31-25-2:~$ docker ps
permission denied while trying to connect to the Docker daemon socket at unix://var/run/docker.sock: Get "http://%2Fvar%2Frun%2Fdocker.sock/v1.24/containers/json": dial unix /var/run/docker.sock: connect: permission denied
ubuntu@ip-172-31-25-2:~$
```

We modify user

Sudo usermod -aG docker \$USER

```
ubuntu@ip-172-31-25-2:~$ sudo usermod -aG docker $USER
```

**Refresh Group:** newgrp docker

```
ubuntu@ip-172-31-25-2:~$ sudo usermod -aG docker $USER && newgrp docker
```

```
ubuntu@ip-172-31-25-2:~$ docker ps
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS   NAMES
ubuntu@ip-172-31-25-2:~$
```

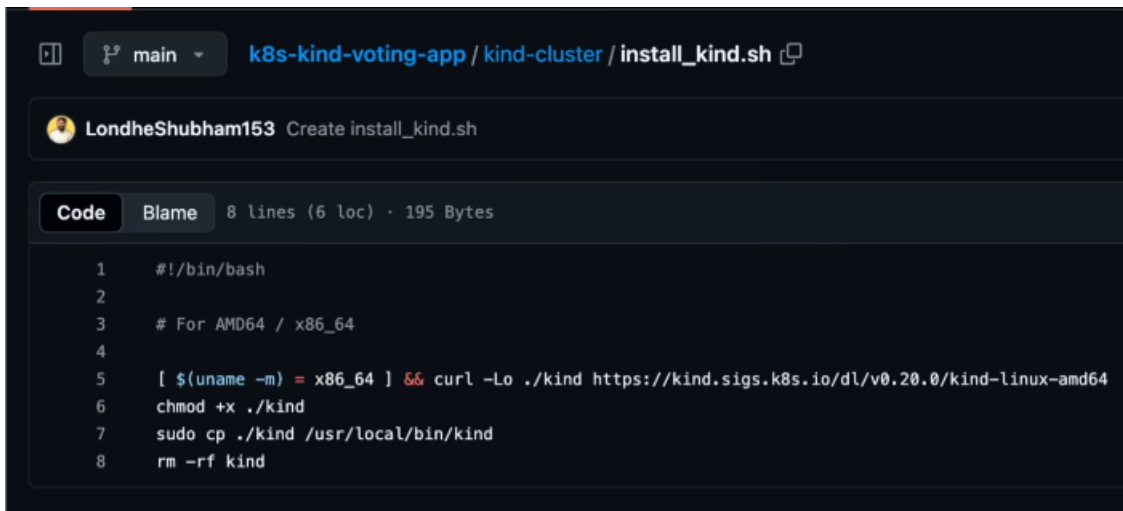
Now we setup Kubernetes Cluster

We see folder called kind-cluster: inside we get all command which help installation

We create Directory k8s-install

```
ubuntu@ip-172-31-25-2:~$ mkdir k8s-install
ubuntu@ip-172-31-25-2:~$ cd k8s-install/
ubuntu@ip-172-31-25-2:~/k8s-install$
```

And create file **vim install\_kind.sh**

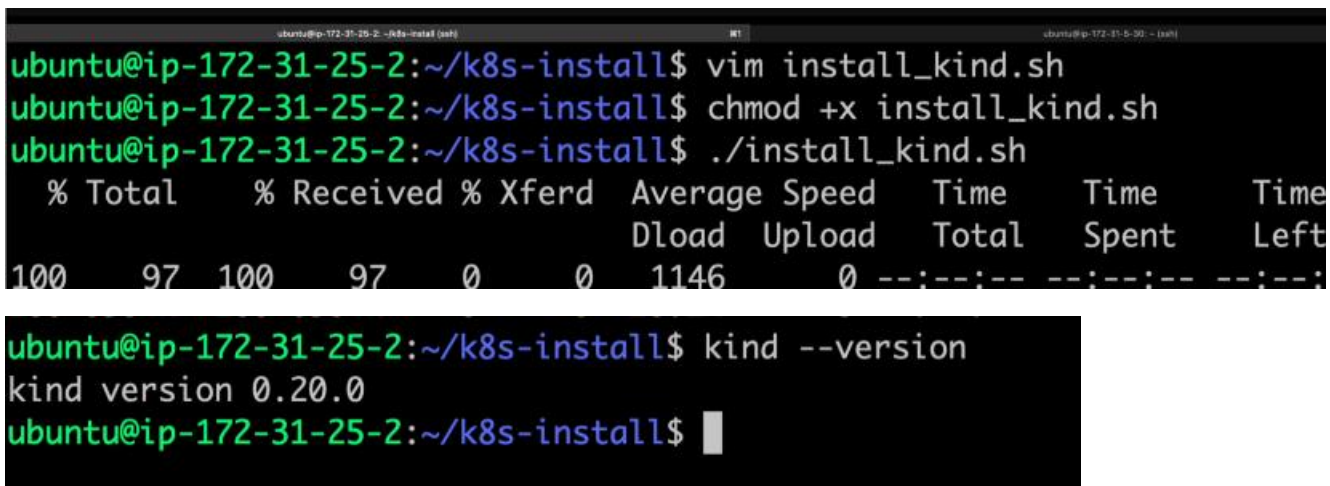


The screenshot shows a GitHub repository for 'k8s-kind-voting-app' with a file named 'install\_kind.sh'. The file is 8 lines long, 6 lines of code, and 195 bytes. The code is as follows:

```
1  #!/bin/bash
2
3  # For AMD64 / x86_64
4
5  [ $(uname -m) = x86_64 ] && curl -Lo ./kind https://kind.sigs.k8s.io/dl/v0.20.0/kind-linux-amd64
6  chmod +x ./kind
7  sudo cp ./kind /usr/local/bin/kind
8  rm -rf kind
```

And we have to make script **executable**

`chmod +x install_kind.sh`



The screenshot shows a terminal session on a Ubuntu machine. The user runs the following commands:

```
ubuntu@ip-172-31-25-2:~/k8s-install$ vim install_kind.sh
ubuntu@ip-172-31-25-2:~/k8s-install$ chmod +x install_kind.sh
ubuntu@ip-172-31-25-2:~/k8s-install$ ./install_kind.sh
```

The output of the script shows a progress bar for downloading the kind binary:

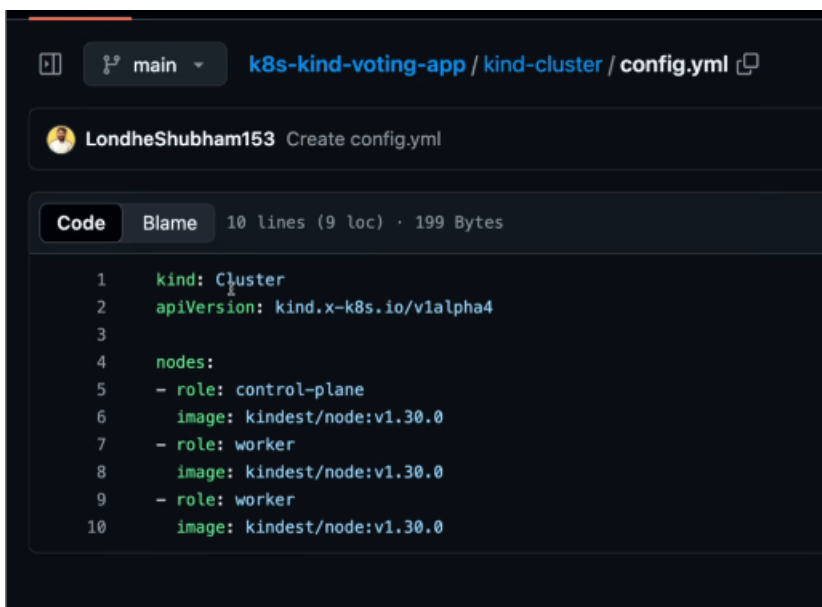
| % Total | % Received | % Xferd | Average Speed | Time   | Time  | Time  |
|---------|------------|---------|---------------|--------|-------|-------|
|         |            |         | Dload         | Upload | Total | Spent |
| 100     | 97         | 100     | 97            | 0      | 0     | 1146  |
|         |            |         |               |        |       | 0     |

The user then runs the command `kind --version` and receives the output:

```
kind version 0.20.0
```

Inside docker you have to **Create cluster master-worker** (you do not need 2 server to create cluster)

This is manifest file Kubernetes , SO we create config.yml file



The screenshot shows a GitHub repository for 'k8s-kind-voting-app' with a file named 'config.yml'. The file is 10 lines long, 9 lines of code, and 199 bytes. The code is as follows:

```
1  kind: Cluster
2  apiVersion: kind.x-k8s.io/v1alpha4
3
4  nodes:
5  - role: control-plane
6    image: kindest/node:v1.30.0
7  - role: worker
8    image: kindest/node:v1.30.0
9  - role: worker
10   image: kindest/node:v1.30.0
```

1 Control plane , 2 Worker node

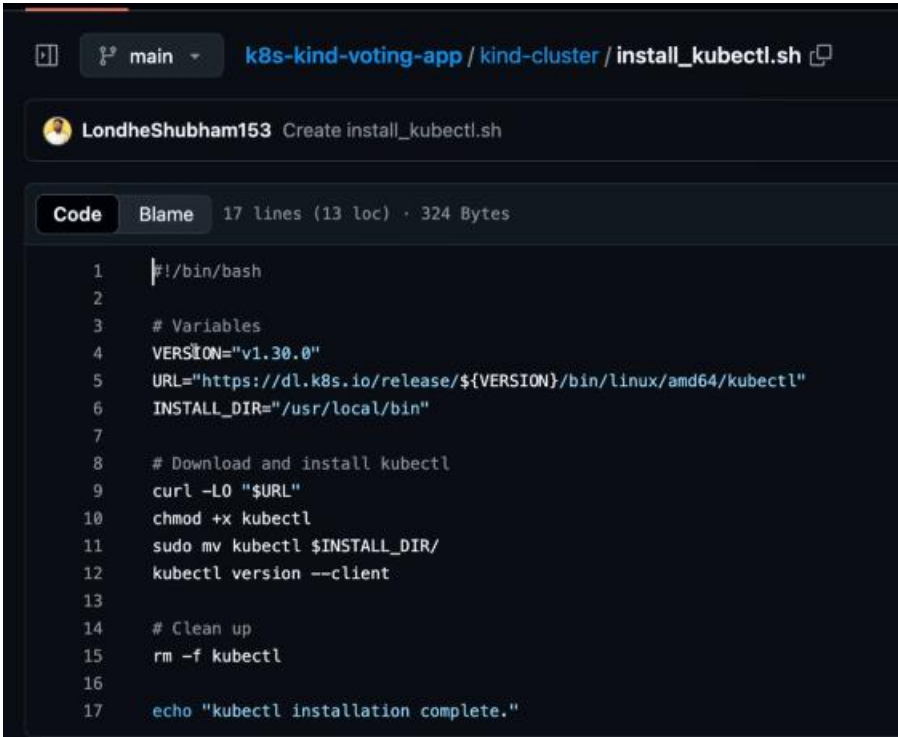
All command you can see **Command.md** file

**Create Cluster** and give name (mycluster)

```
ubuntu@ip-172-31-25-2:~/k8s-install$ kind create cluster --config=config.yml --name=my-cluster
Creating cluster "my-cluster" ...
.: Ensuring node image (kindest/node:v1.30.0) 
```

Now we create Cluster to access it we need Kubectl

**Install kubectl:**



The screenshot shows a GitHub file view for a script named `install_kubectl.sh`. The file is 17 lines long, 13 loc, and 324 Bytes. The code is as follows:

```
1  #!/bin/bash
2
3  # Variables
4  VERSION="v1.30.0"
5  URL="https://dl.k8s.io/release/${VERSION}/bin/linux/amd64/kubectl"
6  INSTALL_DIR="/usr/local/bin"
7
8  # Download and install kubectl
9  curl -LO "$URL"
10  chmod +x kubectl
11  sudo mv kubectl $INSTALL_DIR/
12  kubectl version --client
13
14  # Clean up
15  rm -f kubectl
16
17  echo "kubectl installation complete."
```

```
ubuntu@ip-172-31-25-2:~/k8s-install$ vim install_kubectl.sh
ubuntu@ip-172-31-25-2:~/k8s-install$ chmod +x install_kubectl.sh
ubuntu@ip-172-31-25-2:~/k8s-install$ ./install_kubectl.sh
```

```
ubuntu@ip-172-31-25-2:~/k8s-install$ kubectl get nodes
NAME                                STATUS    ROLES    AGE
my-cluster-control-plane           Ready    control-plane    108s
my-cluster-worker                  Ready    <none>          85s
my-cluster-worker2                  Ready    <none>          85s
ubuntu@ip-172-31-25-2:~/k8s-install$
```

---

Now we setup **ARGO CD** in this cluster

We create Namespace **Argocd**

```
ubuntu@ip-172-31-25-2: ~/k8s-install (ssh)
kubectl create namespace argocd
namespace/argocd created
ubuntu@ip-172-31-25-2:~/k8s-install$
```

```
ubuntu@ip-172-31-25-2:~/k8s-install$ kubectl apply -n argocd -f https://raw.githubusercontent.com/argoproj/argo-cd/stable/manifests/install.yaml
```

```
ubuntu@ip-172-31-25-2:~/k8s-install$ kubectl get pods -n argocd
```

| NAME                                              | RESTARTS | AGE | READY | STATUS   |
|---------------------------------------------------|----------|-----|-------|----------|
| argocd-application-controller-0                   | 0        | 13s | 0/1   | Running  |
| argocd-applicationset-controller-744b76d7fd-w4wx5 | 0        | 14s | 1/1   | Running  |
| argocd-dex-server-5bf5dbc64d-6wx7d                | 0        | 14s | 0/1   | Init:0/1 |
| argocd-notifications-controller-84f5bf6896-db2bq  | 0        | 14s | 1/1   | Running  |
| argocd-redis-74b8999f94-g7rp7                     | 0        | 14s | 0/1   | Init:0/1 |
| argocd-repo-server-57f4899557-rfk9s               | 0        | 14s | 0/1   | Init:0/1 |
| argocd-server-7bc7b97977-pcxm6                    | 0        | 14s | 0/1   | Cr       |

We want to see Argo CD in userinterface for that we have to **check service argocd -server** ( type- **clusterIP** )

We have to make it **NodePort** . So we **Patch it**

• Expose Argo CD server using NodePort:

```
kubectl patch svc argocd-server -n argocd -p '{"spec": {"type": "NodePort"}}'
```

Kubectl get svc --> You see **Argo-cd** server **Service type** is **Nodeport (30680)** . we Do port forwarding 30680 portforwading because we want to expose it

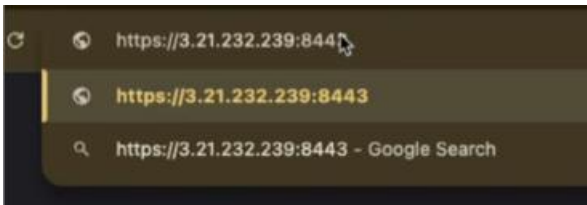
```
argocd-server      NodePort      10.96.110.11
80:30680/TCP,443:31950/TCP  4m21s
```

kubectl port-forward svc/argocd-server -n argocd 8443:443 -- address=0.0.0.0 & < --- & background

```
ubuntu@ip-172-31-25-2:~/k8s-install$ kubectl port-forward -n argocd service/argocd-server 8443:443 --address=0.0.0.0 &
```

Browser: <https://<ip>:8443>

**SG** inbound **8443**, description: argocd server anywhereipv4



Argo cd:

Username: admin

Password:

```
ubuntu@ip-172-31-25-2: ~/k8s-install$ kubectl get secret -n argocd argocd-initial-admin-secret -o jsonpath="{.data.password}" | base64 -d && echo
PnTQWMIrKuIi9F4G
```

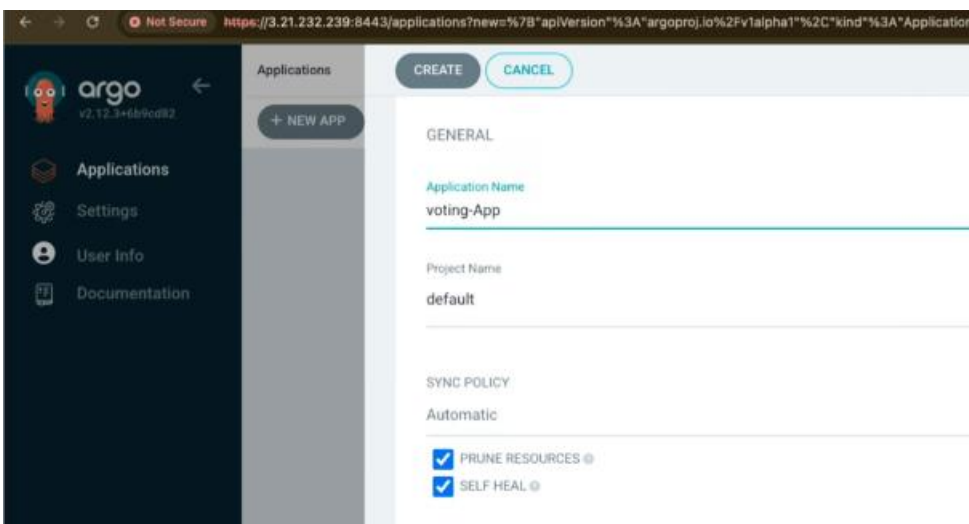
Cluster add + application add + Github Connect deploy

Setting: Cluster : if you same cluster you setup argocd [if you have EKS external cluster than you have to add it here ]

| NAME                                                                                           | URL                            |
|------------------------------------------------------------------------------------------------|--------------------------------|
|  in-cluster | https://kubernetes.default.svc |

Application + New App+ Create

- Prune Resource : old resource delete
- Self Healing: Pod delete crash again up them



We use **Default namespace** no need to create it

Path means you have to mention k8's Manifest file: k8s-specification folder name



## SOURCE

Repository URL

<https://github.com/LondheShubham153/k8s-kind-voting-app.git>

Revision

main

Path

k8s-specifications

## DESTINATION

Cluster URL

<https://kubernetes.default.svc>

Namespace

default

## Create :

Applications / Q voting-app

DETAILS DIFF SYNC SYNC STATUS HISTORY AND ROLLBACK DELETE REFRESH

APP HEALTH **Progressing**

SYNC STATUS **Synced to main (f1d8de2)**

Auto sync is enabled.

Author: Shubham Londhe <shubhamnath5@gmail.com>

Comment: Create commands.md

LAST SYNC **Sync OK to f1d8de2**

Succeeded a few seconds ago (Wed Sep 11 2024 14:42:18 GMT+0530)

Author: Shubham Londhe <shubhamnath5@gmail.com>

Comment: Create commands.md

NAME

NAME

KINDS

KINDS

SYNC STATUS

Synced 9

voting-app

db

redis

result

vote-fb9g9

db-597b4ff8d7

redis-796dc594bb

result-d8c4c69b8

```
ubuntu@ip-172-31-25-2: ~/k8s-install$ kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
db-597b4ff8d7-bmz6z                 1/1     Running   0           35s
redis-796dc594bb-pm4cb              1/1     Running   0           35s
result-d8c4c69b8-cxxw7              1/1     Running   0           35s
vote-69cb46f6fb-f7zqc               1/1     Running   0           35s
vote-69cb46f6fb-jz4rb              1/1     Running   0           35s
```

## Browser Access:

```

ubuntu@ip-172-31-25-2: ~/k8s-install$ kubectl get svc
NAME            TYPE          CLUSTER-IP      EXTERNAL-IP      PORT(S)
db              ClusterIP     10.96.135.184   <none>           5432/TCP
kubernetes      ClusterIP     10.96.0.1       <none>           443/TCP
redis          ClusterIP     10.96.62.192   <none>           6379/TCP
result         NodePort      10.96.3.234     <none>           5001:31001/T
vote           NodePort      10.96.124.78    <none>           5000:31000/T
ubuntu@ip-172-31-25-2:~/k8s-install$

```

Result and Vote ko Port Forward krna hai

```

ubuntu@ip-172-31-25-2:~/k8s-install$ kubectl port-forward svc/vote 5000:5000 --
address=0.0.0.0 &

```

```

ubuntu@ip-172-31-25-2:~/k8s-install$ kubectl port-forward svc/result 5001:5001
--address=0.0.0.0
Forwarding from 0.0.0.0:5001 -> 80

```

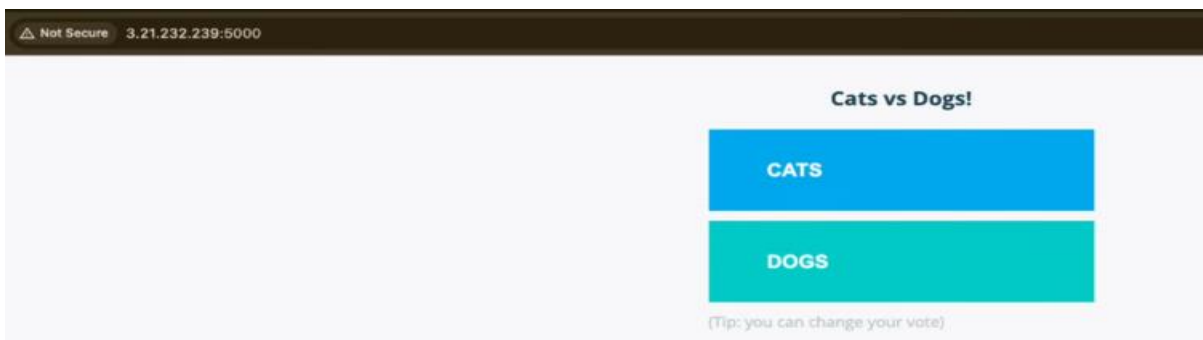
Port range [Info](#)  
  
Description - optional [Info](#)

Source type [Info](#)  

Custom ▲  
Custom ✓  
Anywhere-IPv4  
Anywhere-IPv6  
My IP

Source [Info](#)

Votting App: <ip>;5000



Result: <ip>;5001





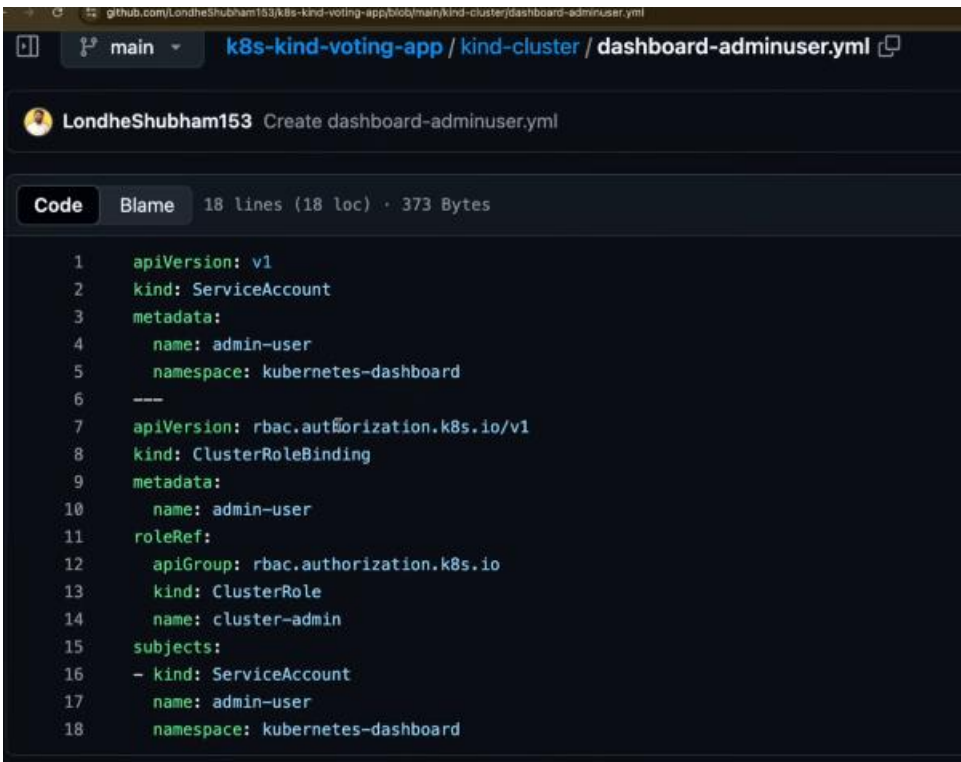
## Kubernetes Dashboard:

### 8. Installing Kubernetes Dashboard

- Deploy Kubernetes dashboard:

```
kubectl apply -f https://raw.githubusercontent.com/kubernetes/dashboard/v2.7.0/aio/deploy/recommended.yaml
```

```
ubuntu@ip-172-31-25-2:~/k8s-install$ kubectl apply -f https://raw.githubusercontent.com/kubernetes/dashboard/v2.7.0/aio/deploy/recommended.yaml
namespace/kubernetes-dashboard created
```



The screenshot shows a GitHub repository for 'k8s-kind-voting-app'. The file 'dashboard-adminuser.yaml' is selected, showing its content. The file is 18 lines long and 373 bytes. It contains two YAML documents: a ServiceAccount named 'admin-user' in the 'kubernetes-dashboard' namespace, and a ClusterRoleBinding named 'admin-user' that binds the 'admin-user' ServiceAccount to the 'cluster-admin' ClusterRole.

```
1  apiVersion: v1
2  kind: ServiceAccount
3  metadata:
4    name: admin-user
5    namespace: kubernetes-dashboard
6  ---
7  apiVersion: rbac.authorization.k8s.io/v1
8  kind: ClusterRoleBinding
9  metadata:
10   name: admin-user
11  roleRef:
12   apiGroup: rbac.authorization.k8s.io
13   kind: ClusterRole
14   name: cluster-admin
15  subjects:
16  - kind: ServiceAccount
17    name: admin-user
18    namespace: kubernetes-dashboard
```

**Service account** (means : user (name admin-user) )

**Create namespace** : kubernetes dashboard

```
ubuntu@ip-172-31-25-2:~/k8s-install$ kubectl create namespace kubernetes-dashbo
ard
```

```
ubuntu@ip-172-31-25-2:~/k8s-install$ kubectl apply -f dashboard.yaml
serviceaccount/admin-user created
clusterrolebinding.rbac.authorization.k8s.io/admin-user created
ubuntu@ip-172-31-25-2:~/k8s-install$
```

Now **Create Token** of that **user** :

- Create a token for dashboard access:
- ```
kubectl -n kubernetes-dashboard create token admin-user
```

Tu access kubernetes dashboard you need token

```
ubuntu@ip-172-31-25-2:~/k8s-install$ kubectl -n kubernetes-dashboard create token  
admin-user
```

COPY

```
ubuntu@ip-172-31-25-2:~/k8s-install$ kubectl -n kubernetes-dashboard create token admin-user
```

To Run **dashboard** you have to **Port forward**

```
ubuntu@ip-172-31-25-2:~/k8s-install$ kubectl get svc -n kubernetes-dashboard
```

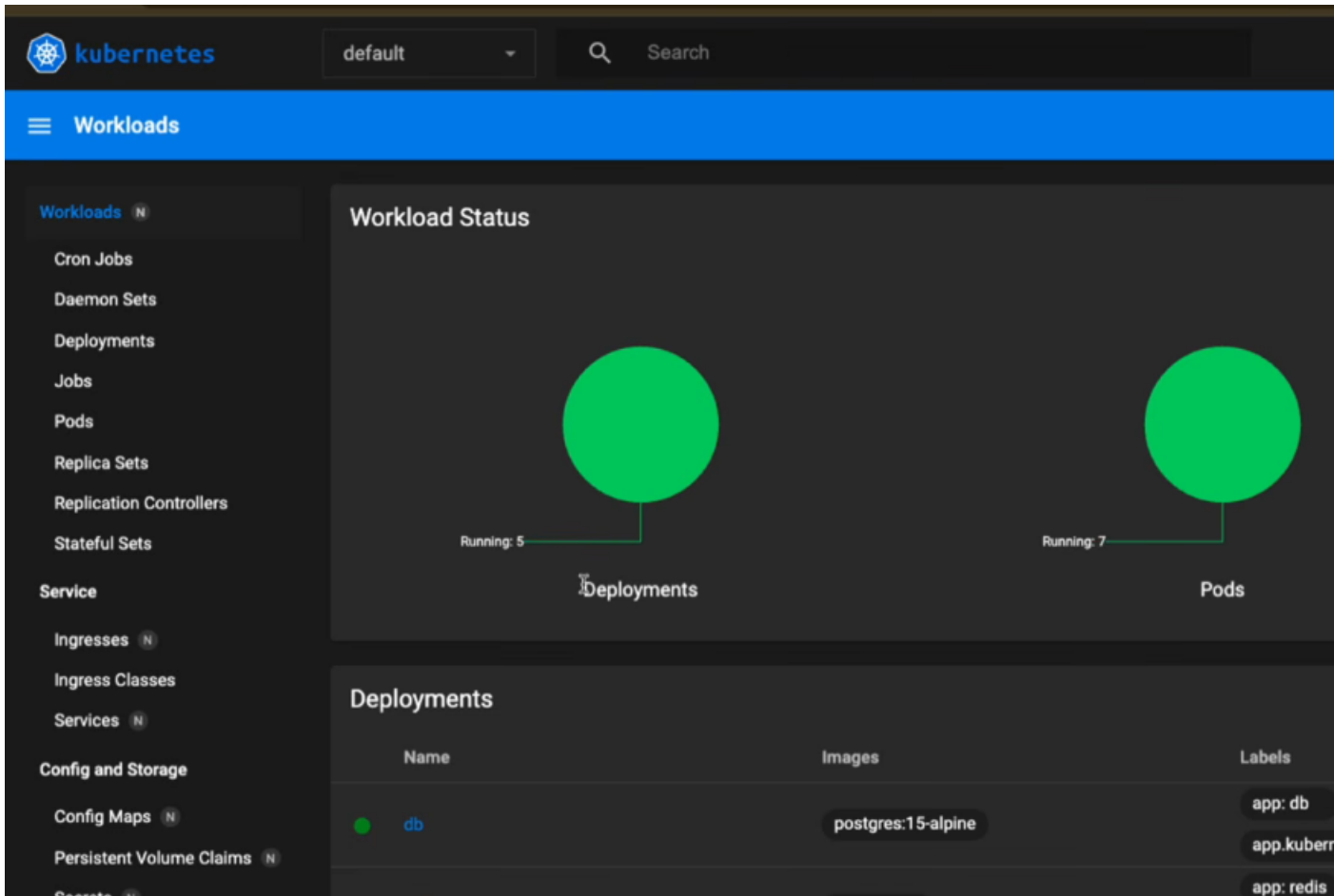
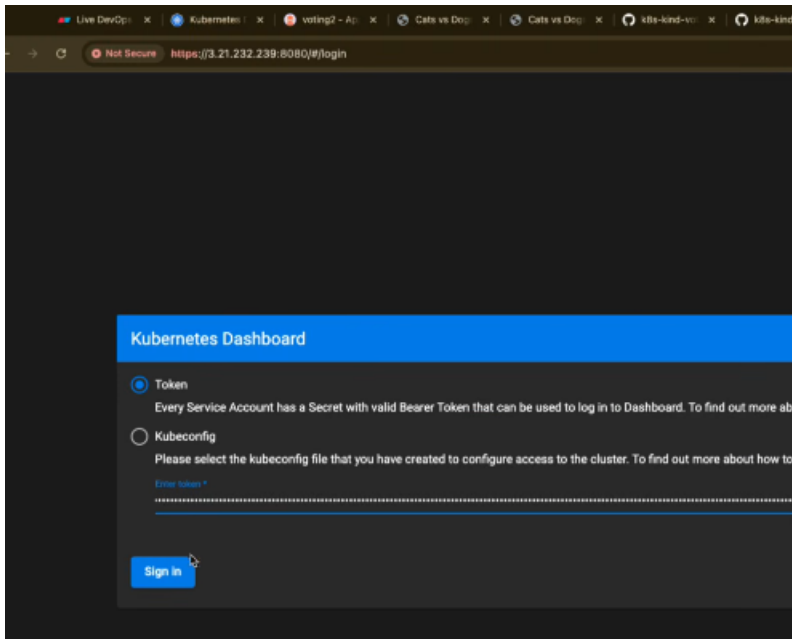
| NAME                      | TYPE      | CLUSTER-IP   | EXTERNAL-IP | PORT(S)  |
|---------------------------|-----------|--------------|-------------|----------|
| dashboard-metrics-scraper | ClusterIP | 10.96.250.15 | <none>      | 8000/TCP |
| kubernetes-dashboard      | ClusterIP | 10.96.161.66 | <none>      | 443/TCP  |

```
ubuntu@ip-172-31-25-2:~/k8s-install$
```

Dashboard run in **443** port forward

```
ubuntu@ip-172-31-25-2:~/k8s-install$ kubectl port-forward svc/kubernetes-dashboard -n kubernetes-dashboard 8080:443 --address=0.0.0.0 &
[5] 10077
ubuntu@ip-172-31-25-2:~/k8s-install$ Forwarding from 0.0.0.0:8080 -> 8443
```

https://<IP>:8080 --> kubernetes dashboard



Complete: